Task 2

Tomorrow's dispatch plan

Take-home assessment

Roughly one-third of your total time budget goes here.

Our field reps visit pharmacies every day to collect orders in person. Each rep works a single area and knows it by heart; what nobody knows by heart is the *order* to run it in. You will change that for one specific day.

Ground rules

1. **Offline code only.** No network calls, no model APIs inside submitted code.
2. **One command reruns everything.** Deterministic, from a clean folder.
3. **Honesty about gaps.** Partial and true beats complete and invented.
4. **Ask, then proceed.** Ambiguity: ask early, assume explicitly, keep moving.
5. **Trust nothing blindly** — including the data, and your own first results.
6. Using AI assistants is allowed and expected; ship an honest `AI_DISCLOSURE.md`. Fabricating results or disclosure ends the conversation.

The data

UTF-8 (BOM) CSVs inside `dataset/`; `_manifest.json` holds checksums.

File	Contents
pharmacy_registry.csv	master record of client pharmacies (location, class, rep, flags)
areas_reference.csv	areas we operate in, with map centers
field_visit_log.csv	what reps report doing, day by day, stop by stop
invoices_app.csv / invoices_erp.csv / invoices_legacy.csv	when every pharmacy actually orders, across three systems

route_plan_tomorrow.csv	tomorrow's requested stops

Your submission

All of your work lives on your fork of this repository (a `task2/` folder with code, `requirements.txt`, an entry point we can run from a clean folder, your README, `AI_DISCLOSURE.md`, and the deliverables below). Commit as you go; the deliverable is your fork link.

The situation

We have **four field employees**, each permanently assigned to one area. Tomorrow morning, **each of them must visit the 16 stops assigned to their area** — `route_plan_tomorrow.csv` names the area, the employee, and the stops. You build the plan for every employee. Fixed conditions: each rep starts and ends the day at their area's map center (see `areas_reference.csv`), lunch is 25 minutes, and everybody must be back by 17:40. Everything else — how long a stop takes, when each pharmacy is best visited, how long the drives really feel — is in the history, if you look for it properly.

What to figure out, in order

1. **How long does anything actually take?** Service time per stop and real drive times, from the visit log's good days. The log is honest about some things and sloppy about others; quantify what is missing or contradictory before trusting any average. Registry coordinates are your map.
2. **When should each stop be visited?** Pharmacies do not order at random hours. Each one has a rhythm; a visit that lands before the rhythm is useful, a visit that lands after it is a wasted drive. Derive per-stop timing targets from the invoices themselves — all three systems count, and not every stop has deep history in every one of them: where the evidence is thin, say so and handle it like a professional instead of inventing a number.
3. **The itinerary, for each employee.** `routing/itinerary.csv`: one plan per area — ordered stops with estimated arrival/departure, drive and service minutes, slack, and the timing target you aimed for at each stop. Prove feasibility against 17:40 with arithmetic a script prints, not prose.
4. **Dispatcher judgment.** Real dispatch days include stops that cannot be served normally. If your preparation reveals any such stop — in any area — handle it the way a good dispatcher would before customers notice: substitute, reorder, or skip with cause, and write down the policy so nobody improvises it again.

Expected output (shape, not answers). one paragraph per affected stop inside `routing/plan.md`: decision, basis, effect on the rest of that area's day.

5. **One-page sensitivity.** For each area: which single-stop failure hurts that day most, and one contingency rule a non-analyst can follow without you.
6. **The route viewer.** A small local web page where we select the area and see that employee's day: the start point, the end point, and the planned path as connected dots, stop by stop, in your planned order. It does not need cartographic realism — dots joined by lines are fine. It does need to reflect YOUR itinerary: when the plan changes, the picture changes.

Expected output (shape, not answers). `itinerary.csv` columns: `area, seq, pharmacy_id, eta, etd, drive_in_min, service_min, slack_min, note` — one block per employee. `routing/plan.md` ($\leq$ 2 pages): the distributions you derived per parameter, the timing targets per stop, and a before/after number showing what any correction you made changed in the plans.

Stack. Streamlit will get you there fastest; any web stack is welcome. Same rules as always: runs locally from your repository with one command, no network calls.

What "good" means here. There is no single correct sequence. A plan that respects each pharmacy's rhythm, is built on history you first made trustworthy, states its assumptions numerically, and survives its own stress test beats a prettier tour built on data you never questioned.